

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-ac9028b4eb07da1e2.jsonl`  
- SHA-256: `603465f3d6bc14afd1d29d5fc5f62535a5ee37edb72a22714f55ef1058cca294`  
- Source modified: `2026-07-04T08:58:54+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_8c732492-138`  
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`
```

Transcript

1. user (2026-07-04T08:53:42.877Z)

You are an ADVERSARIAL verifier for a fix to PR #469 (branch fix/highlights-signed-url) of Khelsutra/badminton-highlight-indexer. A fix agent just left UNCOMMITTED changes in the worktree C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download. Your default stance: try to REFUTE that the fix is correct and complete.

The blocker it must resolve:

REVIEWER BLOCKER (verbatim, from PR #469):

"One more issue: the new storage.signed_url() helper currently ignores the backend's configured signed_url_ttl.

Even though the active storage config is now threaded into signed_reel_url(), the helper signature defaults ttl=3600 and always passes that value into the backend:

```
def signed_url(..., ttl: int = 3600, ...):  
    return get_storage(...).signed_url(ref, ttl=ttl, method=method)
```

Both GCSStorage and S3Storage only fall back to their configured self._ttl when ttl is falsey (ttl or self._ttl), so the configured TTL never gets a chance to apply for /api/highlights.

I reproduced locally on this head with an injected GCS client and config={'gcs': {'signed_url_ttl': 123}}; calling storage.signed_url('gcs://bucket/highlights/vid/reel.mp4', config=..., method='GET') passed an expiration of 3600 seconds, not 123.

Suggested fix: make the helper's ttl optional/None and only pass an explicit ttl when the caller provides one, or pass ttl=0/None through so the backend uses its configured signed_url_ttl.

Please add a small regression test that /api/highlights / signed_reel_url() honors a non-default storage.gcs.signed_url_ttl (and ideally S3 too, since it has the same pattern)."

The fix agent reported:

```
- files changed: ["C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/backend/storage/__init__.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/backend/storage/base.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/backend/storage/gcs.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/backend/storage/s3.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/tests/test_storage.py", "C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download/tests/test_upload_download.py"]  
- tests added: ["tests/test_storage.py :: test_module_signed_url_honors_configured_gcs_ttl", "tests/test_storage.py :: test_module_signed_url_honors_configured_s3_ttl", "tests/test_storage.py :: test_module_signed_url_explicit_ttl_overrides_config", "tests/test_upload_download.py :: test_highlights_signed_url_honors_configured_gcs_ttl"]  
- suite: 1633 passed, 10 skipped, 8 warnings in 126.22s (0:02:06) | ruff: ruff 0.15.16: "All checks passed!" (python -m ruff check ., from worktree root) | mypy: mypy 2.1.0: "Success: no issues found in 147 source files" (python -m mypy backend training; 4 pre-existing annotation-unchecked notes only)  
- summary: Changed the module-level storage.signed_url() helper (backend/storage/__init__.py) from ttl: int = 3600 to ttl: Optional[int] = None so a caller that doesn't pass ttl no longer shadows the operator's configured signed_url_ttl; with ttl=None the backends' existing "ttl or self._ttl" fallback applies. For consistency (and so mypy stays green when the helper passes
```

Optional[int] through the ABC), StorageBackend.signed_url in backend/storage/base.py and the GCSStorage/S3Storage method defaults were changed from ttl=3600 to ttl=None; docstrings on the helper and the ABC now document the None-means-configured-TTL contract citing #469. Audited all callers: the only production caller of any backend .signed_url() is the module helper, and the helper's only production caller is orchestration.signed_reel_url() (backend/orchestration.py:713), which passes no ttl and therefore now honors the configured TTL ? verified end-to-end by a new /api/highlights test that injects a fake GCS client via cfg.storage.gcs and asserts the 307 redirect location carries ttl=123. Added the reviewer's exact GCS repro, an S3 equivalent (ExpiresIn=456), and an explicit-ttl-overrides-config test; the fake GCS blob's generate_signed_url now embeds the expiration seconds in the URL so TTL plumbing is assertable. backend/main.py was not touched (line budget unaffected).
- problems:

DO (read-only on the worktree except where stated):

1. Read the full diff: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download diff (plus git -C C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download status --short for new files). Check the fix logic line by line against the blocker. Hunt for: edge cases the reviewer's repro implies but the fix misses, behavior changes beyond the blocker's scope, style that clashes with the repo, docstrings that overstate.
2. PROVE the regression test bites: create a THROWAWAY detached worktree at the PR head: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download worktree add C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-469 d3b0e64b467ccc243545ddccee9e73e316a5e732 --detach ; copy ONLY the new/changed test file(s) from C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download into it; run those specific tests there with python -m pytest <testfile> -q -p no:cacheprovider ? they MUST FAIL (that proves the test catches the original bug). Then remove the throwaway: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download worktree remove --force C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-469. If the test PASSES at the unfixed head, that is a refutation ? report it.
3. Run the new tests in the REAL worktree to confirm they pass with the fix: python -m pytest <testfile> -q -p no:cacheprovider.
4. Do NOT modify the fix agent's changes in C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download. Do NOT commit or push anywhere. PowerShell tool on Windows; global python 3.13.
5. For PR #468 specifically (if that is your PR): also re-run the new concurrency test 5 times in a row in the real worktree to smoke out flakiness, and report.

Fill the schema honestly. verdict='approve' ONLY if the fix fully resolves the blocker, the test provably bites, and you found no real issues.

2. user (2026-07-04T08:53:49.489Z)

```
M backend/storage/__init__.py
M backend/storage/base.py
M backend/storage/gcs.py
M backend/storage/s3.py
M tests/test_storage.py
M tests/test_upload_download.py
d3b0e64 review: thread active storage config into mirror/exists/sign; extract helper (#463)
504feed fix(cloud-serving): serve large reels via a gs:// signed URL (#463)
a5d6246 serving: re-queue interrupted compile jobs on restart (#329 reliability) (#460)
```

3. user (2026-07-04T08:53:56.094Z)

```
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index a5958cc..38889de 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -206,12 +206,16 @@ def signed_url(
     uri: str,
     *,
     config: Optional[Dict[str, Any]] = None,
-    ttl: int = 3600,
+    ttl: Optional[int] = None,
```

```

        method: str = "GET",
    ) -> str:
        """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from the
store, off
        the app's response path (so a large object isn't streamed back through the server; #463).
Backends
-   without object signing (e.g. local) raise ``NotImplementedError``.
+   without object signing (e.g. local) raise ``NotImplementedError``.
+
+   ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl`` from
+   ``config`` ? a hardcoded default here would silently shadow the operator's setting (#469).
+   An explicit ``ttl`` overrides it for callers that need a specific expiry."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl, method=method)

```

```
diff --git a/backend/storage/base.py b/backend/storage/base.py
```

```
index 74fea15..2a82245 100644
```

```
--- a/backend/storage/base.py
```

```
+++ b/backend/storage/base.py
```

```
@@ -54,9 +54,11 @@ class StorageBackend(ABC):
```

```
    def delete(self, ref: StorageRef) -> None: ...
```

```
    def signed_url(
```

```
-        self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET"
```

```
+        self, ref: StorageRef, *, ttl: "int | None" = None, method: str = "GET"
```

```
) -> str:
```

```
-        """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
```

```
+        """A time-limited URL handing the bytes to a browser/LLM. ``ttl=None`` (the default)
```

```
+        means "use the backend's configured ``signed_url_ttl``" ? so operator config wins
```

```
+        unless a caller explicitly needs a different expiry (#469). Not every backend can
```

```
+        sign (``local`` cannot; Drive only emulates) ? callers must treat
```

```
        ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
```

```
        raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
```

```
diff --git a/backend/storage/gcs.py b/backend/storage/gcs.py
```

```
index 261d72c..a0834ce 100644
```

```
--- a/backend/storage/gcs.py
```

```
+++ b/backend/storage/gcs.py
```

```
@@ -73,7 +73,7 @@ class GCSStorage(StorageBackend):
```

```
    def delete(self, ref) -> None:
```

```
        self._blob(ref).delete()
```

```
-    def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
```

```
+    def signed_url(self, ref, *, ttl=None, method="GET") -> str:
```

```
        blob = self._blob(ref)
```

```
        exp = timedelta(seconds=ttl or self._ttl)
```

```
        creds = getattr(self._client, "_credentials", None)
```

```
diff --git a/backend/storage/s3.py b/backend/storage/s3.py
```

```
index 6c09d15..a9ae7e0 100644
```

```
--- a/backend/storage/s3.py
```

```
+++ b/backend/storage/s3.py
```

```
@@ -87,7 +87,7 @@ class S3Storage(StorageBackend):
```

```
    def delete(self, ref) -> None:
```

```
        self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
```

```
-    def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
```

```
+    def signed_url(self, ref, *, ttl=None, method="GET") -> str:
```

```
        op = "get_object" if method.upper() == "GET" else "put_object"
```

```
        return self._client.generate_presigned_url(
```

```
            op,
```

```
diff --git a/tests/test_storage.py b/tests/test_storage.py
```

```
index 8d7b427..11f1875 100644
```

```
--- a/tests/test_storage.py
```

```
+++ b/tests/test_storage.py
```

```
@@ -250,7 +250,10 @@ class _FakeBlob:
```

```
        self.store.pop((self.bucket, self.name), None)
```

```

    def generate_signed_url(self, version, expiration, method):
-       return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"
+       # Expose the expiration so TTL-plumbing tests can assert the configured
+       # signed_url_ttl actually reaches the signer (#469).
+       ttl = int(expiration.total_seconds())
+       return f"https://gcs.example/{self.bucket}/{self.name}?version={version}&ttl={ttl}"

class _FakeGCSClient:
@@ -309,6 +312,39 @@ def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
    assert not be.exists(ref)

+## ----- module signed_url helper: TTL plumbing
+## (#469)
+## The helper's ttl default must be None so the backend's configured signed_url_ttl applies ?
+## a hardcoded helper default would silently shadow the operator's config for every caller
+## that doesn't pass an explicit ttl (e.g. /api/highlights via signed_reel_url).
+
+def test_module_signed_url_honors_configured_gcs_ttl():
+    # Reviewer repro on #469: config={'gcs': {'signed_url_ttl': 123}} must reach the signer.
+    url = storage.signed_url(
+        "gcs://bucket/highlights/vid/reel.mp4",
+        config={"gcs": {"client": _FakeGCSClient(), "signed_url_ttl": 123}},
+        method="GET",
+    )
+    assert "ttl=123" in url
+
+def test_module_signed_url_honors_configured_s3_ttl():
+    url = storage.signed_url(
+        "s3://bucket/highlights/vid/reel.mp4",
+        config={"s3": {"client": _FakeS3Client(), "signed_url_ttl": 456}},
+    )
+    assert "X-Amz-Expires=456" in url
+
+def test_module_signed_url_explicit_ttl_overrides_config():
+    url = storage.signed_url(
+        "gcs://bucket/x",
+        config={"gcs": {"client": _FakeGCSClient(), "signed_url_ttl": 123}},
+        ttl=999,
+    )
+    assert "ttl=999" in url
+
+    # ----- Google Drive (local mount)
+    # The backend reads a locally MOUNTED Google Drive (Google Drive for Desktop). A tmp_path
+    # folder stands in for the mount's "My Drive" root ? no network, no Drive API, no extra deps.
diff --git a/tests/test_upload_download.py b/tests/test_upload_download.py
index b258a13..14b78d6 100644
--- a/tests/test_upload_download.py
+++ b/tests/test_upload_download.py
@@ -329,6 +329,35 @@ def test_highlights_redirects_to_signed_url_when_reel_in_bucket(env,
monkeypatch
    assert seen.get("sign_cfg") is not None and seen["sign_cfg"].get("output_root") ==
"gs://b/out"

+def test_highlights_signed_url_honors_configured_gcs_ttl(env):
+    """/api/highlights must sign with the operator's storage.gcs.signed_url_ttl, not a
+    hardcoded
+    3600 (#469 review). Exercises the REAL storage layer end-to-end (config ? get_storage ?
+    GCSStorage.signed_url) with an injected fake client ? only the GCS SDK is faked."""

```

```

+
+ class _Blob:
+     def __init__(self, bucket, key):
+         self.bucket, self.key = bucket, key
+
+     def exists(self):
+         return self.key == "out/highlights/vid469/reel.mp4"
+
+     def generate_signed_url(self, version, expiration, method):
+         ttl = int(expiration.total_seconds())
+         return f"https://gcs.example/{self.bucket}/{self.key}?ttl={ttl}"
+
+ class _Client:
+     def bucket(self, name):
+         return type("_B", (), {"blob": lambda _s, key: _Blob(name, key)}}())
+
+ client, db, _tmp, cfg = env
+ cfg.storage.output_root = "gs://b/out"
+ cfg.storage.gcs = {"client": _Client(), "signed_url_ttl": 123}
+ db.add_video("vid469", "gs://b/in/x.mp4", "processed", [])
+ r = client.get("/api/highlights/vid469/reel.mp4", follow_redirects=False)
+ assert r.status_code == 307
+ assert r.headers["location"].endswith("?ttl=123")
+
+
+ def test_highlights_falls_back_to_local_when_no_bucket_reel(env, monkeypatch):
+     """No mirrored bucket object (or a signing hiccup) ? serve the local file, unchanged
+     behaviour."""
+     client, db, _tmp, cfg = env

```

4. user (2026-07-04T08:54:08.777Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import shutil
18    import tempfile
19    from typing import Any, Dict, Iterator, Optional
20
21    from backend.storage.base import StorageBackend
22    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24    __all__ = [
25        "StorageBackend",
26        "StorageRef",
27        "StorageStat",
28        "parse_uri",
29        "resolve_input_uri",

```

```

30     "get_storage",
31     "fetch",
32     "put",
33     "exists",
34     "signed_url",
35     "list_uris",
36     "acquire_local_input",
37     "cleanup_acquired_local_input",
38     "input_is_local",
39     "resolve_input_ref",
40 ]
41
42
43 def _storage_dict(storage_config: Any) -> Dict[str, Any]:
44     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
45     if storage_config is None:
46         return {}
47     if hasattr(storage_config, "model_dump"):
48         return storage_config.model_dump() # type: ignore[no-any-return]
49     return dict(storage_config)
50
51
52 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
53     """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
54     config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
55     callers at the API/CLI boundary catch that and return a clean client error (never a
500).
56
57     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
58     ``local:///`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
59     sc = _storage_dict(storage_config)
60     uri = resolve_input_uri(
61         raw,
62         input_backend=sc.get("input_backend", "local"),
63         input_root=sc.get("input_root", ""),
64     )
65     return parse_uri(uri)
66
67
68 def input_is_local(raw: str, storage_config: Any = None) -> bool:
69     """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
70     plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
71     that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
72     ``resolve_input_ref``)."""
73     return resolve_input_ref(raw, storage_config).backend == "local"
74
75
76 def acquire_local_input(
77     raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
78 ) -> str:
79     """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
80
81     Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
82     byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
83     non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local

```

```

84     path returned. The caller owns successful remote scratch copies; use
85     ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path. """
86     sc = _storage_dict(storage_config)
87     ref = resolve_input_ref(raw, sc)
88     if ref.backend == "local":
89         return (
90             ref.key
91         ) # identity ? no temp, no copy, byte-identical to passing the path directly
92     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
93     try:
94         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
95     except Exception:
96         shutil.rmtree(scratch, ignore_errors=True)
97         raise
98
99
100 def cleanup_acquired_local_input(
101     raw: str, local_path: str, storage_config: Any = None
102 ) -> None:
103     """Delete a remote scratch dir produced by ``acquire_local_input``.
104
105     Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
106     into a fresh ``rally-input-`` directory; deleting that directory is safe once the
caller has
107     finished hashing/validating/stitching/segmenting the returned file.
108     """
109     try:
110         if input_is_local(raw, storage_config):
111             return
112     except ValueError:
113         return
114     if not local_path:
115         return
116     scratch = os.path.dirname(os.path.abspath(local_path))
117     if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
118         shutil.rmtree(scratch, ignore_errors=True)
119
120
121 def _backend_class(name: str):
122     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
123     if name == "local":
124         from backend.storage.local import LocalStorage
125
126         return LocalStorage
127     if name == "s3":
128         from backend.storage.s3 import S3Storage
129
130         return S3Storage
131     if name == "gcs":
132         from backend.storage.gcs import GCSStorage
133
134         return GCSStorage
135     if name == "gdrive":
136         from backend.storage.gdrive import GDriveStorage
137
138         return GDriveStorage
139     if name == "gdrive-api":
140         from backend.storage.gdrive_api import GDriveApiStorage
141
142         return GDriveApiStorage
143     raise ValueError(
144         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
145 )

```

```

146
147
148     def get_storage(
149         backend: Optional[str] = None,
150         *,
151         config: Optional[Dict[str, Any]] = None,
152         **overrides,
153     ) -> StorageBackend:
154         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
155         the implementation; its per-backend settings come from ``config[backend]`` merged
with
156         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
157         cfg = config or {}
158         name = backend or cfg.get("backend", "local")
159         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
160         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
161         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
162         settings: Dict[str, Any] = dict(
163             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
164         )
165         settings.update(overrides)
166         return _backend_class(name)(**settings)
167
168
169     def fetch(
170         uri: str,
171         dst: "str | os.PathLike[str] | None" = None,
172         *,
173         config: Optional[Dict[str, Any]] = None,
174         overwrite: bool = False,
175     ) -> str:
176         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
177         (no copy); for s3/gcs it downloads to ``dst``."""
178         ref = parse_uri(uri)
179         return get_storage(ref.backend, config=config).fetch_to_local(
180             ref, dst, overwrite=overwrite
181         )
182
183
184     def put(
185         local_path: "str | os.PathLike[str]",
186         uri: str,
187         *,
188         config: Optional[Dict[str, Any]] = None,
189         content_type: Optional[str] = None,
190     ) -> str:
191         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
192         ref = parse_uri(uri)
193         get_storage(ref.backend, config=config).put(
194             local_path, ref, content_type=content_type
195         )
196         return uri
197
198
199     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
200         """True if the object at ``uri`` exists (a bucket object or a local file)."""
201         ref = parse_uri(uri)
202         return bool(get_storage(ref.backend, config=config).exists(ref))
203
204

```



```

205     def signed_url(
206         uri: str,
207         *,
208         config: Optional[Dict[str, Any]] = None,
209         ttl: Optional[int] = None,
210         method: str = "GET",
211     ) -> str:
212         """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
213         the app's response path (so a large object isn't streamed back through the server;
#463). Backends
214         without object signing (e.g. local) raise ``NotImplementedError``.
215
216         ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl``
from
217         ``config`` ? a hardcoded default here would silently shadow the operator's setting
(#469).
218         An explicit ``ttl`` overrides it for callers that need a specific expiry."""
219         ref = parse_uri(uri)
220         return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
221
222
223     def list_uris(
224         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
225     ) -> Iterator[str]:
226         """Yield the URI of every object under ``prefix_uri``."""
227         ref = parse_uri(prefix_uri)
228         for r in get_storage(ref.backend, config=config).list(ref):
229             yield r.uri
230

```

5. user (2026-07-04T08:54:09.577Z)

```

C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\orchestration.py:697: GCS
project/client/signed_url_ttl), or None. Threaded into storage calls so a non-default backend
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\orchestration.py:713:
return storage.signed_url(reel_uri, config=settings, method="GET")
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\__init__.py:34:
"signed_url",
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\__init__.py:205: def
signed_url(
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\__init__.py:216:
``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl`` from
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\__init__.py:220:
return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl, method=method)
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:25:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:28:
self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:76: def
signed_url(self, ref, *, ttl=None, method="GET") -> str:
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:82: # so
a plain generate_signed_url raises "you need a private key". Fall back to the IAM signBlob
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:94:
return blob.generate_signed_url(
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\gcs.py:103:
return blob.generate_signed_url(version="v4", expiration=exp, method=method)
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\base.py:56: def
signed_url(
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\base.py:60:
means "use the backend's configured ``signed_url_ttl``" ? so operator config wins
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\s3.py:30:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khehsutra-guru\wt-signed-download\backend\storage\s3.py:33:

```

```

self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:90:    def
signed_url(self, ref, *, ttl=None, method="GET") -> str:
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:92:
return self._client.generate_presigned_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\docs\PLATFORM_ARCHITECTURE.md:119:
def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\docs\PLATFORM_ARCHITECTURE.md:138:
(for compute) and `signed_url` (for the provider/browser), so the pipeline stops caring
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:98: def
test_local_signed_url_raises():
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:100:
LocalStorage().signed_url(parse_uri("/x"))
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:197:    def
generate_presigned_url(self, op, Params, ExpiresIn):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:216:    url =
be.signed_url(ref, ttl=900)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:252:    def
generate_signed_url(self, version, expiration, method):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:254:        #
signed_url_ttl actually reaches the signer (#469).
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:310:    assert
be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:315: #
----- module signed_url helper: TTL plumbing
(#469)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:316: # The
helper's ttl default must be None so the backend's configured signed_url_ttl applies ?
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:321: def
test_module_signed_url_honors_configured_gcs_ttl():
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:322:    #
Reviewer repro on #469: config={'gcs': {'signed_url_ttl': 123}} must reach the signer.
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:323:    url =
storage.signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:325:
config={"gcs": {"client": _FakeGCSClient(), "signed_url_ttl": 123}},
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:331: def
test_module_signed_url_honors_configured_s3_ttl():
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:332:    url =
storage.signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:334:
config={"s3": {"client": _FakeS3Client(), "signed_url_ttl": 456}},
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:339: def
test_module_signed_url_explicit_ttl_overrides_config():
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:340:    url =
storage.signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:342:
config={"gcs": {"client": _FakeGCSClient(), "signed_url_ttl": 123}},
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:419: def
test_gdrive_signed_url_not_supported(tmp_path):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:422:
GDriveStorage(mount_root=str(_mount(tmp_path))).signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:305: def
test_highlights_redirects_to_signed_url_when_reel_in_bucket(env, monkeypatch):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:322:
monkeypatch.setattr(m.storage, "signed_url", _signed)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:326:
# the active storage settings are threaded into BOTH exists() and signed_url() (so a non-default
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:327:
# backend / signed_url_ttl is honored), not a fresh default client.
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:332: def
test_highlights_signed_url_honors_configured_gcs_ttl(env):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:333:
"""/api/highlights must sign with the operator's storage.gcs.signed_url_ttl, not a hardcoded
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:335:

```

```
GCSStorage.signed_url) with an injected fake client ? only the GCS SDK is faked.""
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:344:
def generate_signed_url(self, version, expiration, method):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:354:
cfg.storage.gcs = {"client": _Client(), "signed_url_ttl": 123}
```

6. user (2026-07-04T08:54:18.772Z)

```
80         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
81             for obj in page.get("Contents", []) or []:
82                 key = obj["Key"]
83                 yield StorageRef(
84                     "s3", prefix.bucket, key, f"s3://{prefix.bucket}/{key}"
85                 )
86
87     def delete(self, ref) -> None:
88         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
89
90     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
91         op = "get_object" if method.upper() == "GET" else "put_object"
92         return self._client.generate_presigned_url(
93             op,
94             Params={"Bucket": ref.bucket, "Key": ref.key},
95             ExpiresIn=ttl or self._ttl,
96         )
97
```

7. user (2026-07-04T08:54:20.132Z)

No matches found

8. user (2026-07-04T08:54:27.164Z)

```
15     from backend.storage.base import StorageBackend
16     from backend.storage.ref import StorageRef, StorageStat
17
18
19     class GCSStorage(StorageBackend):
20         def __init__(
21             self,
22             *,
23             client: Any = None,
24             project: Optional[str] = None,
25             signed_url_ttl: int = 3600,
26             **_ignored,
27         ) -> None:
28             self._ttl = signed_url_ttl
29             if client is not None:
30                 self._client = client
31             return
32             storage_module: Any = import_module("google.cloud.storage")
33             self._client = storage_module.Client(project=project)
34
35         def _blob(self, ref: StorageRef):
36             return self._client.bucket(ref.bucket).blob(ref.key)
37
38         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39             if dst is None:
40                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
41             import os
42
43             dst = str(dst)
44             if os.path.exists(dst) and not overwrite:
45                 return dst
```

```

46         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
47         self._blob(ref).download_to_filename(dst)
48         return dst
49
50     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
51         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
52         return ref
53
54     def exists(self, ref) -> bool:
55         return bool(self._blob(ref).exists())
56
57     def stat(self, ref) -> StorageStat:
58         b = self._blob(ref)
59         b.reload()
60         return StorageStat(
61             size=int(b.size or 0),
62             modified=b.updated,
63             etag=b.etag,
64             content_type=b.content_type,
65         )
66
67     def list(self, prefix) -> Iterator[StorageRef]:
68         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
69             yield StorageRef(
70                 "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
71             )
72
73     def delete(self, ref) -> None:
74         self._blob(ref).delete()
75
76     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
77         blob = self._blob(ref)
78         exp = timedelta(seconds=ttl or self._ttl)
79         creds = getattr(self._client, "_credentials", None)
80         # Local-key signing (a service-account JSON key exposes a `signer`) works out of
the box. But
81         # on Cloud Run / GCE the ADC credentials come from the metadata server and have
NO private key,
82         # so a plain generate_signed_url raises "you need a private key". Fall back to
the IAM signBlob
83         # API by passing the SA email + a fresh access token ? which requires the
runtime SA to hold
84         # roles/iam.serviceAccountTokenCreator on ITSELF. Best-effort: if we can't build
that, defer to
85         # the default call (which will raise a clear error rather than silently mis-
sign).
86         if creds is not None and not hasattr(creds, "signer"):
87             try:
88                 from google.auth.transport.requests import Request as _AuthRequest
89
90                 if not getattr(creds, "token", None):
91                     creds.refresh(_AuthRequest())
92                 sa_email = getattr(creds, "service_account_email", None)
93                 if sa_email and sa_email != "default":
94                     return blob.generate_signed_url(
95                         version="v4",
96                         expiration=exp,
97                         method=method,
98                         service_account_email=sa_email,
99                         access_token=creds.token,
100                     )
101             except Exception: # pragma: no cover - exercised only on real Cloud Run
creds
102                 pass
103         return blob.generate_signed_url(version="v4", expiration=exp, method=method)

```

9. assistant (2026-07-04T08:54:39.228Z)

The fix logic checks out. Now check base.py for annotation-style consistency, then prove the tests bite at the unfixed head.

10. user (2026-07-04T08:54:40.103Z)

```

1      """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3      Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4      live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5      pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      """
8
9      from __future__ import annotations
10
11     import os
12     from abc import ABC, abstractmethod
13     from typing import Iterator
14
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class StorageBackend(ABC):
19         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
20         worker dispatcher needs; the rest support listing/checkpointing/serving."""
21
22         @abstractmethod
23         def fetch_to_local(
24             self,
25             ref: StorageRef,
26             dst: "str | os.PathLike[str] | None" = None,
27             *,
28             overwrite: bool = False,
29         ) -> str:
30             """Make the object available as a local file; return its local path. For the
31             ``local`` backend this is an identity passthrough (the object is already
local)."""
32
33         @abstractmethod
34         def put(
35             self,
36             local_path: "str | os.PathLike[str]",
37             ref: StorageRef,
38             *,
39             content_type: "str | None" = None,
40         ) -> StorageRef:
41             """Upload/copy a local file to ``ref``; return ``ref``."""
42
43         @abstractmethod
44         def exists(self, ref: StorageRef) -> bool: ...
45
46         @abstractmethod
47         def stat(self, ref: StorageRef) -> StorageStat: ...
48
49         @abstractmethod
50         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
51             """Yield a ``StorageRef`` per object under ``prefix``."""
52

```

```

53     @abstractmethod
54     def delete(self, ref: StorageRef) -> None: ...
55
56     def signed_url(
57         self, ref: StorageRef, *, ttl: "int | None" = None, method: str = "GET"
58     ) -> str:
59         """A time-limited URL handing the bytes to a browser/LLM. ``ttl=None`` (the
default)
60         means "use the backend's configured ``signed_url_ttl``" ? so operator config
wins
61         unless a caller explicitly needs a different expiry (#469). Not every backend
can
62         sign (``local`` cannot; Drive only emulates) ? callers must treat
63         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."
64         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
65

```

11. user (2026-07-04T08:54:47.095Z)

```

685     def reel_object_uri(cfg, video_id: str, name: str) -> str:
686         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
687         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
688         mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."
689         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
690         if not root or "://" not in root:
691             return ""
692         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
693
694
695     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
696         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
697         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
698         isn't silently constructed as a fresh default."""
699         sc = getattr(cfg, "storage", None)
700         return sc.model_dump() if sc is not None else None
701
702
703     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
704         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
705         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
706         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
707         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
708         reel_uri = reel_object_uri(cfg, video_id, name)
709         if not reel_uri:
710             return None
711         settings = _storage_settings(cfg)
712         if storage.exists(reel_uri, config=settings):
713             return storage.signed_url(reel_uri, config=settings, method="GET")
714         return None
715
716
717     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
718         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
719         and return the {status, filepath, download_url} result the SPA polls for. Returns a

```

12. user (2026-07-04T08:54:58.854Z)

Preparing worktree (detached HEAD d3b0e64)
HEAD is now at d3b0e64 review: thread active storage config into mirror/exists/sign; extract helper (#463)

13. user (2026-07-04T08:55:14.466Z)

Exit code 1

FF.F

[100%]

===== FAILURES =====

_____ test_module_signed_url_honors_configured_gcs_ttl _____

```
def test_module_signed_url_honors_configured_gcs_ttl():
    # Reviewer repro on #469: config={'gcs': {'signed_url_ttl': 123}} must reach the signer.
    url = storage.signed_url(
        "gcs://bucket/highlights/vid/reel.mp4",
        config={"gcs": {"client": _FakeGCSClient(), "signed_url_ttl": 123}},
        method="GET",
    )
> assert "ttl=123" in url
E     AssertionError: assert 'ttl=123' in
'https://gcs.example/bucket/highlights/vid/reel.mp4?version=v4&ttl=3600'
```

tests\test_storage.py:328: AssertionError

_____ test_module_signed_url_honors_configured_s3_ttl _____

```
def test_module_signed_url_honors_configured_s3_ttl():
    url = storage.signed_url(
        "s3://bucket/highlights/vid/reel.mp4",
        config={"s3": {"client": _FakeS3Client(), "signed_url_ttl": 456}},
    )
> assert "X-Amz-Expires=456" in url
E     AssertionError: assert 'X-Amz-Expires=456' in
'https://s3.example/bucket/highlights/vid/reel.mp4?X-Amz-Expires=3600'
```

tests\test_storage.py:336: AssertionError

_____ test_highlights_signed_url_honors_configured_gcs_ttl _____

```
env = (<starlette.testclient.TestClient object at 0x0000018CE15BE3C0>,
<backend.infrastructure.database.Database object at 0..., margin=0.0),
flywheel=FlywheelConfig(enabled=False, workspace_prefix='flywheel', shadow_eval=True,
min_promote_n=5)))
```

```
def test_highlights_signed_url_honors_configured_gcs_ttl(env):
    """/api/highlights must sign with the operator's storage.gcs.signed_url_ttl, not a
    hardcoded
    3600 (#469 review). Exercises the REAL storage layer end-to-end (config ? get_storage ?
    GCSStorage.signed_url) with an injected fake client ? only the GCS SDK is faked."""
```

```
class _Blob:
```

```
    def __init__(self, bucket, key):
        self.bucket, self.key = bucket, key
```

```
    def exists(self):
        return self.key == "out/highlights/vid469/reel.mp4"
```

```
    def generate_signed_url(self, version, expiration, method):
        ttl = int(expiration.total_seconds())
        return f"https://gcs.example/{self.bucket}/{self.key}?ttl={ttl}"
```

```
class _Client:
```

```
    def bucket(self, name):
        return type("_B", (), {"blob": lambda _s, key: _Blob(name, key)})(())
```

```

client, db, _tmp, cfg = env
cfg.storage.output_root = "gs://b/out"
cfg.storage.gcs = {"client": _Client(), "signed_url_ttl": 123}
db.add_video("vid469", "gs://b/in/x.mp4", "processed", [])
r = client.get("/api/highlights/vid469/reel.mp4", follow_redirects=False)
assert r.status_code == 307
> assert r.headers["location"].endswith("?ttl=123")
E     AssertionError: assert False
E       + where False = <built-in method endswith of str object at
0x00000018CE1694AB0>('?ttl=123')
E       +   where <built-in method endswith of str object at 0x00000018CE1694AB0> =
'https://gcs.example/b/out/highlights/vid469/reel.mp4?ttl=3600'.endswith

tests\test_upload_download.py:358: AssertionError
===== short test summary info =====
FAILED tests/test_storage.py::test_module_signed_url_honors_configured_gcs_ttl
FAILED tests/test_storage.py::test_module_signed_url_honors_configured_s3_ttl
FAILED tests/test_upload_download.py::test_highlights_signed_url_honors_configured_gcs_ttl
3 failed, 1 passed in 3.05s

```

14. user (2026-07-04T08:55:31.128Z)

```

..... [100%]
59 passed in 3.48s

```

15. user (2026-07-04T08:55:42.594Z)

All checks passed!

```

backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 147 source files

```

16. user (2026-07-04T08:58:04.477Z)

```

.....SS..... [ 4%]
.....S..... [ 8%]
..... [ 13%]
..... [ 17%]
..... [ 21%]
..... [ 26%]
..... [ 30%]
..... [ 35%]
..... [ 39%]
..... [ 43%]
..... [ 48%]
..... [ 52%]
..... [ 57%]
..... [ 61%]
..... [ 65%]
..... [ 70%]
.....SSSSS..... [ 74%]
..... [ 78%]
..... [ 83%]
..... [ 87%]
..... [ 92%]
..... [ 96%]
..... [100%]
===== warnings summary =====

```



```
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_endpoints.py::test_process_nonlocal_uri_skips_exists_and_fetches
tests/test_api_endpoints.py::test_process_remote_input_reports_use_durable_output_dir
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gs://bucket/videos/x.mp4]
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gs://bucket/videos/x.mp4]
```

```
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1633 passed, 10 skipped, 8 warnings in 131.76s (0:02:11)
```

17. assistant (2026-07-04T08:58:26.636Z)

Full suite, ruff, and mypy all verified green locally. One last check ? whether the repo has a coverage floor in CI config.

18. user (2026-07-04T08:58:28.234Z)

No matches found

19. user (2026-07-04T08:58:54.428Z)

Structured output provided successfully